

AI Assisted Coding

Assignment Lab - 5.1 and 6.1

Name : P. Gouri Prasad Varma

Hall.t.No. : 2303A51035

Batch No. : 01

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Lab Objectives:

- To explore the ethical risks associated with AI-generated code.
- To recognize issues related to security, bias, transparency, and copyright.
- To reflect on the responsibilities of developers when using AI tools in software development.
- To promote awareness of best practices for responsible and ethical AI coding.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Identify and avoid insecure coding patterns generated by AI tools.
- Detect and analyze potential bias or discriminatory logic in AI-generated outputs.
- Evaluate originality and licensing concerns in reused AI-generated code.
- Understand the importance of explainability and transparency in AI-assisted programming.
- Reflect on accountability and the human role in ethical AI coding practices.

Task 1:

Employee Data: Create Python code that defines a class named `Employee` with the following attributes: `empid`, `empname`, `designation`, `basic\_salary`, and `exp`. Implement a method `display\_details()` to print all employee details. Implement another method `calculate\_allowance()` to determine additional allowance

based on experience:

- If `exp > 10 years` → allowance = 20% of `basic\_salary`
- If `5 ≤ exp ≤ 10 years` → allowance = 10% of `basic\_salary`
- If `exp < 5 years` → allowance = 5% of `basic\_salary`

Finally, create at least one instance of the `Employee` class, call the `display\_details()` method, and print the calculated allowance.

Copilot Suggested Code :

```
# Employee data

class Employee:
    def __init__(self, emp_id, emp_name, designation, emp_salary, emp_exp):
        self.emp_id = emp_id
        self.emp_name = emp_name
        self.designation = designation
        self.emp_salary = emp_salary
        self.emp_exp = emp_exp
    def display_details(self):
        print(f"Employee ID: {self.emp_id}")
        print(f"Employee Name: {self.emp_name}")
        print(f"Designation: {self.designation}")
        print(f"Salary: {self.emp_salary}")
        print(f"Experience: {self.emp_exp} years")
    def calculate_allowance(self):
        if self.emp_exp > 10:
            allowance = 0.1 * self.emp_salary
        elif 5 < self.emp_exp <= 10:
            allowance = 0.05 * self.emp_salary
        else:
            allowance = 0.005 * self.emp_salary
        print(f"Allowance: {allowance}")

emp = Employee(101, "John", "Manager", 15000, 12)
emp.display_details()
emp.calculate_allowance()
emp2 = Employee(102, "Alice", "Developer", 8000, 7)
emp2.display_details()
emp2.calculate_allowance()
```

Output :

```
PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 1.py"
Employee ID: 101
Employee Name: John
Designation: Manager
Salary: 15000
Experience: 12 years
Allowance: 1500.0
Employee ID: 102
Employee Name: Alice
Designation: Developer
Salary: 8000
Experience: 7 years
Allowance: 400.0
PS C:\Btech\3 - 2\Ai AC> █
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print customer details, and a method `calculate\_bill()` where:

- Units $\leq 100 \rightarrow ₹5$ per unit
- 101 to 300 units $\rightarrow ₹7$ per unit
- More than 300 units $\rightarrow ₹10$ per unit

Create a bill object, display details, and print the total bill amount.

Copilot Suggested Code :

```
1  # Electricity Bill
2
3  class ElectricityBill:
4      def __init__(self, cus_id, name, units_consumed):
5          self.cus_id = cus_id
6          self.name = name
7          self.units_consumed = units_consumed
8          self.bill_amount = 0
9      def display_details(self):
10         print(f"Customer ID: {self.cus_id}")
11         print(f"Customer Name: {self.name}")
12         print(f"Units Consumed: {self.units_consumed}")
13         print(f"Bill Amount: {self.bill_amount}")
14     def calculate_bill(self):
15         if self.units_consumed <= 100:
16             self.bill_amount = self.units_consumed * 5
17         elif 101 <= self.units_consumed <= 300:
18             self.bill_amount = 100 * 5 + (self.units_consumed - 100) * 7
19         else:
20             self.bill_amount = 100 * 5 + 200 * 7 + (self.units_consumed - 300) * 10
21         print(f"Total Bill Amount: ₹{self.bill_amount}")
22
23     bill = ElectricityBill(1001, "Rahul", 350)
24     bill.display_details()
25     bill.calculate_bill()
26
27     bill2 = ElectricityBill(1002, "Priya", 250)
28     bill2.display_details()
29     bill2.calculate_bill()
```

Output :

```
PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 2.py"
Customer ID: 1001
Customer Name: Rahul
Units Consumed: 350
Bill Amount: 0
Total Bill Amount: ₹2400
Customer ID: 1002
Customer Name: Priya
Units Consumed: 250
Bill Amount: 0
Total Bill Amount: ₹1550
PS C:\Btech\3 - 2\Ai AC>
```

Task 3:

Product Discount Calculation- Create Python code that defines a class named `Product` with attributes: `product\_id`, `product\_name`, `price`, and `category`. Implement a method `display\_details()` to print product details.

Implement another method

`calculate\_discount()` where:

- Electronics → 10% discount
- Clothing → 15% discount
- Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.

Copilot Suggested Code :

```
1  # Product Discount Calculation
2
3  class Product:
4      def __init__(self, product_id, product_name, category):
5          self.product_id = product_id
6          self.product_name = product_name
7          self.category = category
8          self.discount = 0
9
10     def display_details(self):
11         print(f"Product ID: {self.product_id}")
12         print(f"Product Name: {self.product_name}")
13         print(f"Category: {self.category}")
14         print(f"Discount: {self.discount}%")
15
16     def calculate_discount(self):
17         if self.category == "Electronics":
18             self.discount = 10
19         elif self.category == "Clothing":
20             self.discount = 15
21         elif self.category == "Grocery":
22             self.discount = 5
23
24     print(f"Final price after discount: {self.discount}%")
25
26 Product1 = Product(1, "Laptop", "Electronics")
27 Product1.display_details()
28 Product1.calculate_discount()
29
30 Product2 = Product(2, "T-Shirt", "Clothing")
31 Product2.display_details()
32 Product2.calculate_discount()
```

Output :

```
PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 3.py"
Product ID: 1
Product Name: Laptop
Category: Electronics
Discount: 0%
Final price after discount: 10%
Product ID: 2
Product Name: T-Shirt
Category: Clothing
Discount: 0%
Final price after discount: 15%
PS C:\Btech\3 - 2\Ai AC> █
```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named `LibraryBook` with attributes: `book\_id`, `title`, `author`, `borrower`, and `days\_late`. Implement a method `display\_details()` to print book details, and a method `calculate\_late\_fee()` where:

- Days late $\leq 5 \rightarrow ₹5$ per day
- 6 to 10 days late $\rightarrow ₹7$ per day
- More than 10 days late $\rightarrow ₹10$ per day

Create a book object, display details, and print the late fee.

Copilot Suggested Code :

```
1  # Book Late Fee Calculation
2
3  class LibraryBook:
4      def __init__(self, book_id, title, author, borrower, days_late):
5          self.book_id = book_id
6          self.title = title
7          self.author = author
8          self.borrower = borrower
9          self.days_late = days_late
10         self.late_fee = 0
11
12     def display_details(self):
13         print(f"Book ID: {self.book_id}")
14         print(f"Title: {self.title}")
15         print(f"Author: {self.author}")
16         print(f"Borrower: {self.borrower}")
17         print(f"Days Late: {self.days_late}")
18         print(f"Late Fee: ₹{self.late_fee}")
19
20     def calculate_late_fee(self):
21         if self.days_late <= 5:
22             self.late_fee = self.days_late * 5
23         elif self.days_late <= 10:
24             self.late_fee = 5 * 5 + (self.days_late - 5) * 7
25         else:
26             self.late_fee = 5 * 5 + 5 * 7 + (self.days_late - 10) * 10
27         print(f"Late Fee calculated: ₹{self.late_fee}")
28
29     book = LibraryBook(101, "The Great Gatsby", "F. Scott Fitzgerald", "John Doe", 7)
30     book.display_details()
31     book.calculate_late_fee()
32
33     book2 = LibraryBook(102, "To Kill a Mockingbird", "Harper Lee", "Jane Smith", 12)
34     book2.display_details()
35     book2.calculate_late_fee()
```

Output :

```
● PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 4.py"
Book ID: 101
Title: The Great Gatsby
Author: F. Scott Fitzgerald
Borrower: John Doe
Days Late: 7
Late Fee: ₹0
Late Fee calculated: ₹39
Book ID: 102
Title: To Kill a Mockingbird
Author: Harper Lee
Borrower: Jane Smith
Days Late: 12
Late Fee: ₹0
Late Fee calculated: ₹80
○ PS C:\Btech\3 - 2\Ai AC> █
```

Task 5:

Student Performance Report - Define a function

`student\_report(student\_data)` that accepts a dictionary containing student names and their marks.

The function should:

- Calculate the average score for each student
- Determine pass/fail status (pass ≥ 40)
- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the output.

Copilot Suggested Code :

```
1  # Student Performance Report
2
3  def student_report(student_data):
4      report = []
5      for name, marks in student_data.items():
6          average = sum(marks) / len(marks)
7          status = "Pass" if average >= 40 else "Fail"
8          report.append({
9              "name": name,
10             "average": average,
11             "status": status
12         })
13     return report
14
15     student_data = {
16         "Alice": [85, 92, 78],
17         "Bob": [35, 45, 50],
18         "Charlie": [90, 88, 92],
19         "Diana": [40, 42, 38]
20     }
21
22     report = student_report(student_data)
23     print(report)
```

Output :

```
PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 5.py"
[{'name': 'Alice', 'average': 85.0, 'status': 'Pass'}, {'name': 'Bob', 'average': 43.333333333333336, 'status': 'Pass'}, {'name': 'Charlie', 'average': 90.0, 'status': 'Pass'}, {'name': 'Diana', 'average': 40.0, 'status': 'Pass'}]
PS C:\Btech\3 - 2\Ai AC>
```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named `TaxiRide` with attributes: `ride\_id`, `driver\_name`, `distance\_km`, and `waiting\_time\_min`. Implement a method `display\_details()` to print ride details, and a method `calculate\_fare()` where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

Copilot Suggested Code :

```
1  # Taxi Fare Calculation
2
3  class TaxiRide:
4      def __init__(self, ride_id, driver_name, distance_km, waiting_time_minutes):
5          self.ride_id = ride_id
6          self.driver_name = driver_name
7          self.distance_km = distance_km
8          self.waiting_time_minutes = waiting_time_minutes
9
10     def display_details(self):
11         print(f"Ride ID: {self.ride_id}")
12         print(f"Driver Name: {self.driver_name}")
13         print(f"Distance (km): {self.distance_km}")
14         print(f"Waiting Time (minutes): {self.waiting_time_minutes}")
15
16     def calculate_fare(self):
17         fare = 0
18         if self.distance_km <= 10:
19             fare += self.distance_km * 15
20         elif self.distance_km <= 30:
21             fare += 10 * 15 + (self.distance_km - 10) * 12
22         else:
23             fare += 10 * 15 + 20 * 12 + (self.distance_km - 30) * 10
24         fare += self.waiting_time_minutes * 2
25
26         print(f"Total Fare: ₹{fare}")
27
28     ride = TaxiRide("T001", "Ramesh", 35, 15)
29     ride.display_details()
30     ride.calculate_fare()
31
32     ride2 = TaxiRide("T002", "Suresh", 25, 10)
33     ride2.display_details()
34     ride2.calculate_fare()
```

Output :

```
PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 6.py"
Ride ID: T001
Driver Name: Ramesh
Distance (km): 35
Waiting Time (minutes): 15
Total Fare: ₹470
Ride ID: T002
Driver Name: Suresh
Distance (km): 25
Waiting Time (minutes): 10
Total Fare: ₹350
PS C:\Btech\3 - 2\Ai AC> █
```

Task 7:

Statistics Subject Performance - Create a Python function

`statistics\_subject(scores\_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic.

Copilot Suggested Code :

```
1  # Statistics Subject Performance
2
3  def statistics_subject(scores_list):
4      if not scores_list:
5          return None
6      highest = max(scores_list)
7      lowest = min(scores_list)
8      average = sum(scores_list) / len(scores_list)
9      passed = sum(1 for score in scores_list if score >= 40)
10     failed = sum(1 for score in scores_list if score < 40)
11     return {
12         "highest": highest,
13         "lowest": lowest,
14         "average": average,
15         "passed": passed,
16         "failed": failed
17     }
18
19 print(statistics_subject([85, 92, 78, 96, 88, 73, 91, 84, 89, 76]))
```

Output :

```
PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 7.py"
{'highest': 96, 'lowest': 73, 'average': 85.2, 'passed': 10, 'failed': 0}
PS C:\Btech\3 - 2\Ai AC>
```

Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements.

Copilot Suggested Code :

Naive Approach :

```
# Naive Approach
def is_prime_naive(n):
    """Check if a number is prime using the naive approach."""
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Optimized Approach :

```
# Optimized Approach
def is_prime_optimized(n):
    """Check if a number is prime using the optimized approach."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
```

Output :

```
# Example Usage
print(is_prime_naive(29))      # True
# Statistics Subject Performance
print(is_prime_optimized(100)) # False
```

Explanation of Optimization:

- The optimized version improves performance by reducing the number of checks needed to determine if a number is prime.
- Instead of checking all numbers up to $n-1$, it only checks up to the square root of n .
- Additionally, it skips even numbers and multiples of 3 after handling them separately, which significantly reduces the number of iterations for larger numbers.

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

Copilot Suggested Code :

```
1  # Recursive function to calculate Fibonacci numbers with explanation
2
3  def fibonacci(n):
4      """
5          Calculate the nth Fibonacci number using recursion.
6
7          Parameters:
8          n (int): The position in the Fibonacci sequence to calculate.
9
10         Returns:
11         int: The nth Fibonacci number.
12         """
13         # Base case: the first Fibonacci number is 0
14         if n == 0:
15             return 0
16         # Base case: the second Fibonacci number is 1
17         elif n == 1:
18             return 1
19         else:
20             # Recursive case: sum of the two preceding Fibonacci numbers
21             return fibonacci(n - 1) + fibonacci(n - 2)
22
23     # Example usage
24     print(fibonacci(10)) # Output: 55
```

Output :

```
● PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 9.py"
55
○ PS C:\Btech\3 - 2\Ai AC> |
```

Explanation of Recursion in the Fibonacci Function

Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem. In the case of the Fibonacci function:

1. Base Cases: The function has two base cases:

- When n is 0, it returns 0.
- When n is 1, it returns 1.

These base cases are essential because they provide stopping points for the recursion, preventing infinite loops.

2. Recursive Calls: For any n greater than 1, the function calls itself twice:

- It calls `fibonacci( $n - 1$ )` to get the $(n-1)$ th Fibonacci number.
- It calls `fibonacci( $n - 2$ )` to get the $(n-2)$ th Fibonacci number.

The results of these two calls are then summed to produce the n th Fibonacci number.

3. Execution Flow: When `fibonacci(10)` is called, the function will keep breaking down the problem into smaller Fibonacci calculations until it reaches the base cases. The results are then combined as the recursive calls return back up the call stack.

This explanation matches the actual execution of the code, as each call to `fibonacci( $n$ )` results in further calls until the base cases are reached, at which point the results are combined to yield the final output.

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior

Copilot Suggested Code :

```
1  # "Generate code with proper error handling and clear explanations for each exception"
2  # Python program to read a file and process data with error handling
3
4  def read_and_process_file(file_path):
5      """
6      Reads a file and processes its data.
7
8      Parameters:
9      file_path (str): The path to the file to be read.
10
11     Returns:
12     list: A list of processed data lines.
13     """
14     processed_data = []
15     try:
16         # Attempt to open the file
17         with open(file_path, 'r') as file:
18             for line in file:
19                 # Process each line (for example, stripping whitespace)
20                 processed_line = line.strip()
21                 processed_data.append(processed_line)
22     except FileNotFoundError:
23         # Handle the case where the file does not exist
24         print(f"Error: The file at '{file_path}' was not found.")
25     except PermissionError:
26         # Handle the case where there are permission issues
27         print(f"Error: Permission denied when trying to read '{file_path}'.")
28     except Exception as e:
29         # Handle any other exceptions that may occur
30         print(f"An unexpected error occurred: {e}")
31     else:
32         # If no exceptions were raised, return the processed data
33         return processed_data
34
35 # Example usage
36 file_path = 'C:\\Btech\\3 - 2\\Ai AC\\Assignment - 5.1 & 6.1\\data.txt'
37 data = read_and_process_file(file_path)
38 if data is not None:
39     print("Processed Data:")
40     for line in data:
41         print(line)
```

Output :

```
PS C:\Btech\3 - 2\Ai AC> & C:\Python314\python.exe "c:/Btech/3 - 2/Ai AC/Assignment - 5.1 & 6.1/Task - 10.py"
Processed Data:
Hello I'm a CS student studying in SR University.
PS C:\Btech\3 - 2\Ai AC>
```

Explanation of Error Handling in the Code

1. `FileNotFoundError`: This exception is raised when the specified file does not exist at the given path. The code catches this exception and prints a user-friendly message indicating that the file was not found.

2. `PermissionError`: This exception occurs when the program does not have the necessary permissions to read the file. The code handles this by informing the user about the permission issue.

3. General Exception Handling: The generic Exception catch-all is used to handle any other unforeseen errors that may arise during file operations. This ensures that the program does not crash unexpectedly and provides feedback about the error.

4. Else Clause: The else clause is executed only if no exceptions were raised in the try block. This is where the processed data is returned, ensuring that the function only returns valid data when the file operations are successful.

This explanation aligns with the runtime behavior of the code, as each specific exception is caught and handled appropriately, providing clarity on what went wrong during file processing.